

BÀI 9. VALIDATION DỮ LIỆU

Bài trước, chúng ta đã xây dựng xong các form CRUD. Tuy nhiên, các form này vẫn còn khá "ngây thơ": chúng chấp nhận bất kỳ dữ liệu nào người dùng nhập vào, kể cả việc bỏ trống tên sản phẩm hay nhập giá tiền là số âm. Điều này có thể gây ra lỗi và làm hỏng dữ liệu. Bài này, chúng ta sẽ dạy cho ứng dụng cách trở thành một **"người bảo vệ"** nghiêm ngặt cho CSDL của mình bằng **Validation**.

MỤC TIÊU BÀI HỌC

Sau khi hoàn thành Bài học này, bạn sẽ có khả năng:

- **Hiểu được tầm quan trọng cốt lõi** của việc xác thực dữ liệu đầu vào.
- **Sử dụng thành thạo Data Annotations** để định nghĩa các quy tắc nghiệp vụ ngay trên Model.
- **Vận dụng ModelState.IsValid** như một "chốt chặn an ninh" không thể thiếu trong các Action [HttpPost].
- **Tích hợp Validation Tag Helpers** để hiển thị các thông báo lỗi một cách thân thiện và trực quan cho người dùng.

NỘI DUNG CHI TIẾT

9.1. Giới thiệu Input Validation - Nguyên tắc "Rác đầu vào, Rác đầu ra"

9.1.1. Nguyên tắc GIGO (Garbage In, Garbage Out)

Đây là một trong những nguyên tắc cơ bản và tàn nhẫn nhất trong lập trình. Nó có nghĩa là: Nếu bạn cho phép dữ liệu không hợp lệ (rác) đi vào hệ thống, thì kết quả bạn nhận lại và lưu trữ cũng sẽ là rác.

- Ví dụ trong SportsStore:
 - Nếu người dùng có thể tạo sản phẩm với giá là -100.000đ, logic tính tổng giỏ hàng của bạn sẽ sai.
 - Nếu người dùng có thể tạo sản phẩm không có tên, trang web của bạn sẽ hiển thị những khoảng trống xấu xí và khó hiểu.
 - **Validation** là quá trình kiểm tra, xác thực dữ liệu đầu vào để đảm bảo nó tuân

thủ các quy tắc đã định **trước khi** được xử lý hoặc lưu trữ. Nó là **người lính gác cổng** cho CSDL của bạn.

9.1.2. Validation phía Server vs. Phía Client: "An ninh vòng trong" và "Lễ tân vòng ngoài"

Có hai nơi chúng ta có thể đặt "chốt chặn":

– Client-Side Validation (Phía Client - "Lễ tân vòng ngoài"):

- **Là gì?** Kiểm tra được thực hiện ngay trên **trình duyệt của người dùng** bằng JavaScript.
- **Mục đích:** Cung cấp **phản hồi tức thì** cho người dùng mà không cần gửi dữ liệu lên server. Ví dụ: báo lỗi ngay khi người dùng bỏ trống một ô bắt buộc.
- **Ưu điểm:** Nhanh, cải thiện trải nghiệm người dùng.
- **Nhược điểm: KHÔNG AN TOÀN.** Người dùng có thể dễ dàng vô hiệu hóa JavaScript hoặc dùng các công cụ như Postman để gửi dữ liệu trực tiếp lên server, **vượt qua hoàn toàn** chốt chặn này.

– Server-Side Validation (Phía Server - "An ninh vòng trong"):

- **Là gì?** Kiểm tra được thực hiện trên **server** bằng code C# sau khi dữ liệu đã được gửi đi.
- **Mục đích:** Đảm bảo **tính toàn vẹn và an toàn** cho dữ liệu. Đây là tuyến phòng thủ cuối cùng và quan trọng nhất.
- **Ưu điểm: AN TOÀN TUYỆT ĐỐI.** Không thể bị qua mặt.
- **Nhược điểm:** Chậm hơn vì cần một vòng lặp request-response.

Kết luận: Validation phía client là để cải thiện trải nghiệm người dùng. Validation phía server là để đảm bảo an ninh và sự đúng đắn của dữ liệu. **Validation phía server là bắt buộc, không bao giờ được bỏ qua.**

9.2. Validation với Data Annotations - Viết "Luật" ngay trên Model

ASP.NET Core cung cấp một cách cực kỳ thanh lịch để định nghĩa các quy tắc validation: **Data Annotations**.

9.2.1. Data Annotations là gì?

Đó là các **attribute** (được đặt trong cặp ngoặc vuông [...]) mà bạn "trang trí" trực tiếp lên các thuộc tính của lớp Model. Chúng giống như những "nhãn dán" chứa đựng các quy tắc.

- **[Required]**: Thuộc tính này không được phép null hoặc là chuỗi rỗng.
- **[Range(min, max)]**: Đối với kiểu số, giá trị phải nằm trong khoảng cho phép.
- **[StringLength(max, MinimumLength = min)]**: Giới hạn độ dài của chuỗi.
- **[RegularExpression("pattern")]**: Giá trị phải khớp với một biểu thức chính quy (regular expression).
- **ErrorMessage = "..."**: Một thuộc tính chung của tất cả các annotation, cho phép bạn tùy chỉnh thông báo lỗi sẽ hiển thị cho người dùng.

9.2.2. ModelState - "Cuốn sổ ghi lỗi" của người gác cổng

Khi một request [HttpPost] được gửi đến, cơ chế Model Binding sẽ cố gắng điền dữ liệu vào model của bạn. Trong quá trình đó, nó sẽ đọc các Data Annotation.

- Nếu một quy tắc bị vi phạm (ví dụ, Name bị bỏ trống trong khi có [Required]), Model Binder sẽ **ghi lại một lỗi** vào một đối tượng đặc biệt tên là ModelState.
- ModelState giống như một cuốn sổ ghi chép, chứa một danh sách tất cả các lỗi validation đã xảy ra trong request hiện tại.

9.2.3. ModelState.IsValid - Chốt chặn an ninh

Trong Action của Controller, bạn có một công cụ cực kỳ quyền lực: thuộc tính ModelState.IsValid.

- Thuộc tính này sẽ trả về true nếu "cuốn sổ ghi lỗi" ModelState **trống trơn**.
- Nó sẽ trả về false nếu có **ít nhất một lỗi** được ghi lại.

Đây chính là "chốt chặn an ninh" mà chúng ta đặt ở đầu mỗi Action [HttpPost].

9.2.4. Áp dụng vào SportsStore: Thêm các annotation vào Product.cs

```
//C#  
// SportsStore.Domain/Product.cs  
  
// BẮT BUỘC phải có using này để sử dụng các Data Annotation
```

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace SportsStore.Domain;

public class Product
{
    public int ProductID { get; set; }

    // [Required]: Báo cho ASP.NET Core rằng trường này là bắt buộc.
    // ErrorMessage: Thông báo sẽ được hiển thị cho người dùng nếu quy tắc
    này bị vi phạm.
    [Required(ErrorMessage = "Please enter a product name")]
    public required string Name { get; set; }

    [Required(ErrorMessage = "Please enter a description")]
    public required string Description { get; set; }

    [Required]
    // [Range]: Áp dụng cho kiểu số. Giá trị phải nằm trong khoảng từ 0.01
    đến vô cực.
    [Range(0.01, double.MaxValue, ErrorMessage = "Please enter a positive
    price")]
    // [Column]: Cho EF Core biết cách ánh xạ thuộc tính này vào CSDL.
    // Ở đây, chúng ta chỉ định kiểu dữ liệu trong CSDL là decimal với độ
    chính xác 18 chữ số, 2 chữ số sau dấu phẩy.
    [Column(TypeName = "decimal(18, 2)")]
    public decimal Price { get; set; }

    [Required(ErrorMessage = "Please specify a category")]
    public required string Category { get; set; }

    public string? ImageUrl { get; set; }
}
```

9.3. Validation Tag Helpers và Client-Side Validation

Sau khi server phát hiện lỗi, làm thế nào để báo cho người dùng một cách thân thiện? Đó là nhiệm vụ của **Validation Tag Helpers**.

9.3.1.asp-validation-for - "Người đưa tin" cá nhân

Đây là một Tag Helper được đặt ngay bên dưới một thẻ `<input>`.

- **Nó làm gì?** Nó "lắng nghe" ModelState. Nếu ModelState chứa lỗi cho thuộc tính mà nó được gán (`asp-validation-for="Name"`), nó sẽ tự động hiển thị ErrorMessage tương ứng vào bên trong thẻ `<span>` của nó.

– **Ví dụ:**

```
//HTML
<div class="form-group">
  <label asp-for="Name"></label>
  <input asp-for="Name" class="form-control" />
  <span asp-validation-for="Name" class="text-danger"></span>
</div>
```

9.3.2. asp-validation-summary - "Bảng tin" tổng hợp

Tag Helper này thường được đặt ở đầu form.

- **Nó làm gì?** Nó có thể hiển thị một danh sách tóm tắt tất cả các lỗi trong ModelState.

– **Ví dụ:**

```
//HTML
<div asp-validation-summary="All" class="text-danger"></div>
```

9.3.3. Bật "Lễ tân vòng ngoài" (Client-Side Validation)

Điều tuyệt vời nhất là các Data Annotation bạn viết ở backend có thể được **tự động sử dụng** để tạo ra các quy tắc validation phía client bằng JavaScript.

- **Cách hoạt động:** Khi ASP.NET Core render một thẻ `<input>` có Tag Helper `asp-for`, nó sẽ nhìn vào các Data Annotation của thuộc tính đó.
 - Thấy `[Required]`, nó sẽ thêm attribute `data-val-required="Thông báo lỗi"` vào thẻ HTML.
 - Thấy `[Range(1, 100)]`, nó sẽ thêm `data-val-range-min="1"` và `data-val-range-max="100"`.
- **Kích hoạt:** Bạn chỉ cần thêm các file script của thư viện **jQuery Unobtrusive Validation** vào trang của bạn (thường là trong `_Layout.cshtml`). Các script này sẽ tự động đọc các attribute `data-val-*` đó và áp dụng các quy tắc validation của

jQuery mà bạn không cần phải viết một dòng JavaScript nào!

9.4. Hướng dẫn thực hành

9.4.1. Thêm các Attribute vào Product.cs

- **Thao tác:** Mở file Product.cs trong project SportsStore.Domain.
- Thêm `using System.ComponentModel.DataAnnotations;` và các attribute như trong phần ví dụ code ở trên.

9.4.2. Thêm các Tag Helper vào Edit.cshtml

- **Thao tác:** Mở file Edit.cshtml trong Views/Admin.
- **Code:**

```
//HTML
@model Product
@{
    ViewBag.Title = "Edit Product";
    Layout = "_AdminLayout";
}

<div asp-validation-summary="ModelOnly" class="text-danger"></div>

<form asp-action="Edit" method="post" enctype="multipart/form-data">
    <input type="hidden" asp-for="ProductID" />
    <div class="form-group">
        <label asp-for="Name"></label>
        <input asp-for="Name" class="form-control" />
        <span asp-validation-for="Name" class="text-danger"></span>
    </div>

    <div class="form-group">
        <label asp-for="Price"></label>
        <input asp-for="Price" class="form-control" />
        <span asp-validation-for="Price" class="text-danger"></span>
    </div>

    <div class="text-center">
        <button class="btn btn-primary" type="submit">Save</button>
    </div>
</form>
```

```
<a asp-action="Index" class="btn btn-secondary">Cancel</a>
</div>
</form>
```

CÂU HỎI LÝ THUYẾT

1. Tại sao chúng ta không thể chỉ tin tưởng vào validation phía client?
2. ModelState.IsValid sẽ trả về true hay false nếu người dùng bỏ trống một trường được đánh dấu [Required]?
3. Sự khác biệt giữa asp-validation-for và asp-validation-summary là gì?
4. Để client-side validation hoạt động tự động, ASP.NET Core render ra các attribute HTML đặc biệt nào?

TỔNG KẾT BÀI HỌC

Bài này, chúng ta đã trang bị cho ứng dụng một "hệ thống miễn dịch" mạnh mẽ:

- Chúng ta đã hiểu rõ tầm quan trọng của việc **không bao giờ tin tưởng dữ liệu đầu vào**.
- Chúng ta đã học cách định nghĩa các "**luật chơi**" (**quy tắc validation**) một cách tập trung và rõ ràng ngay trên Model bằng **Data Annotations**.
- Chúng ta đã biết cách sử dụng **ModelState.IsValid** như một **chốt chặn an ninh** trên server.
- Chúng ta đã học cách cung cấp **phản hồi lỗi thân thiện** cho người dùng bằng **Validation Tag Helpers**.

Ứng dụng của bạn giờ đây không chỉ hoạt động, mà còn hoạt động một cách **an toàn và đáng tin cậy**.

BÀI TẬP VỀ NHÀ

1. **Hoàn thiện Validation:** Áp dụng các Tag Helper span asp-validation-for cho tất cả các trường còn lại trong Edit.cshtml.
2. **Kích hoạt Client-Side Validation:**
 - Mở file _AdminLayout.cshtml (hoặc _Layout.cshtml nếu bạn đang dùng chung).
 - Thêm các dòng script sau vào cuối file, ngay trước thẻ </body>:

```
//HTML
<script src="~/lib/jquery/dist/jquery.min.js"></script>
<script src="~/lib/jquery-validation/dist/jquery.validate.min.js"></script>
<script src="~/lib/jquery-validation-
unobtrusive/jquery.validate.unobtrusive.min.js"></script>
```

Các file này thường có sẵn trong wwwroot/lib khi tạo project MVC mới. Nếu không, bạn cần thêm chúng vào.

- Chạy lại ứng dụng, vào trang Edit, thử bỏ trống một ô và bấm sang ô khác. Bạn sẽ thấy lỗi hiện ra ngay lập tức!
3. **(Nâng cao)** Tìm hiểu và áp dụng thêm một Data Annotation khác vào Product.cs, ví dụ: [StringLength(500, MinimumLength = 10, ErrorMessage = "Description must be between 10 and 500 characters")] cho thuộc tính Description.

Tài liệu tham khảo

- [1]. **Model validation in ASP.NET Core MVC:** [Microsoft Docs - Model Validation](#)
- [2]. **System.ComponentModel.DataAnnotations Namespace:** [Microsoft Docs - DataAnnotations](#)
- [3]. **The Validation Tag Helpers:** [Microsoft Docs - Validation Tag Helpers](#)